Concurrent File Processing Pipeline — Implementation Doc

Oct 30, 2026 · @Muskan

Overview and goals

We build a Python service (FastAPI + Celery) where a browser uploads a batch of files (target: 50 per batch, up to 100 MB each) straight to S3 using presigned URLs. Celery workers then extract data from each file and deliver it to one third-party vendor that accepts at most 20 concurrent calls and takes **10–20 seconds** to respond. S3 and the vendor are mocked so the whole system runs locally with `docker compose up`.

Stack

Concern	Choice	Why
API	FastAPI + Uvicorn, Pydantic v2	Async endpoints, SSE support, typed request/response models
Task queue	Celery 5.4, Redis broker	Mature retries, routing, beat scheduler; Redis also holds limiter and breaker state
Extract workers	Celery `prefork` pool	Parsing is CPU-bound; processes avoid the GIL
Deliver workers	Celery `gevent` pool	Vendor calls wait 10–20 s on I/O; greenlets hold hundreds of waits cheaply
Periodic jobs	Celery beat	Outbox relay, sweeper, stuck-upload cleanup
Database	Postgres 16, SQLAlchemy 2.0, Alembic	Source of truth for task state
S3 / HTTP clients	boto3, httpx	boto3 works against MinIO unchanged; httpx has explicit timeouts
Mock vendor	FastAPI app, random 10–20 s delay per call	Realistic slow vendor with a 20-call limit and fault injection

Concern	Choice	Why
Client	React + TypeScript	Browser-side upload has to run in JavaScript
Tests and ops	pytest, moto, testcontainers, Playwright, Locust, Flower, structlog, OpenTelemetry	

Must have

1. Files go from the client directly to S3. The API server never touches file bytes.
2. Failures are contained per file and per vendor. One bad file never fails the batch, and a vendor outage only pauses delivery, and every failure ends in a visible, retryable state.
3. Every uploaded file gets a **task id** at batch creation, usable to track that file from upload to vendor delivery.
4. Users see live progress for each task and the batch, and can opt in to a notification when all files are processed.
5. On failure, the user can see the task's state, the stage it failed in, the cause, the attempts made, and what happens next.
6. Bursts from many users are queued and processed continuously: the vendor's 20-call limit is never exceeded, its capacity is shared fairly between users, and no request is rejected because the vendor is busy.

Design goals

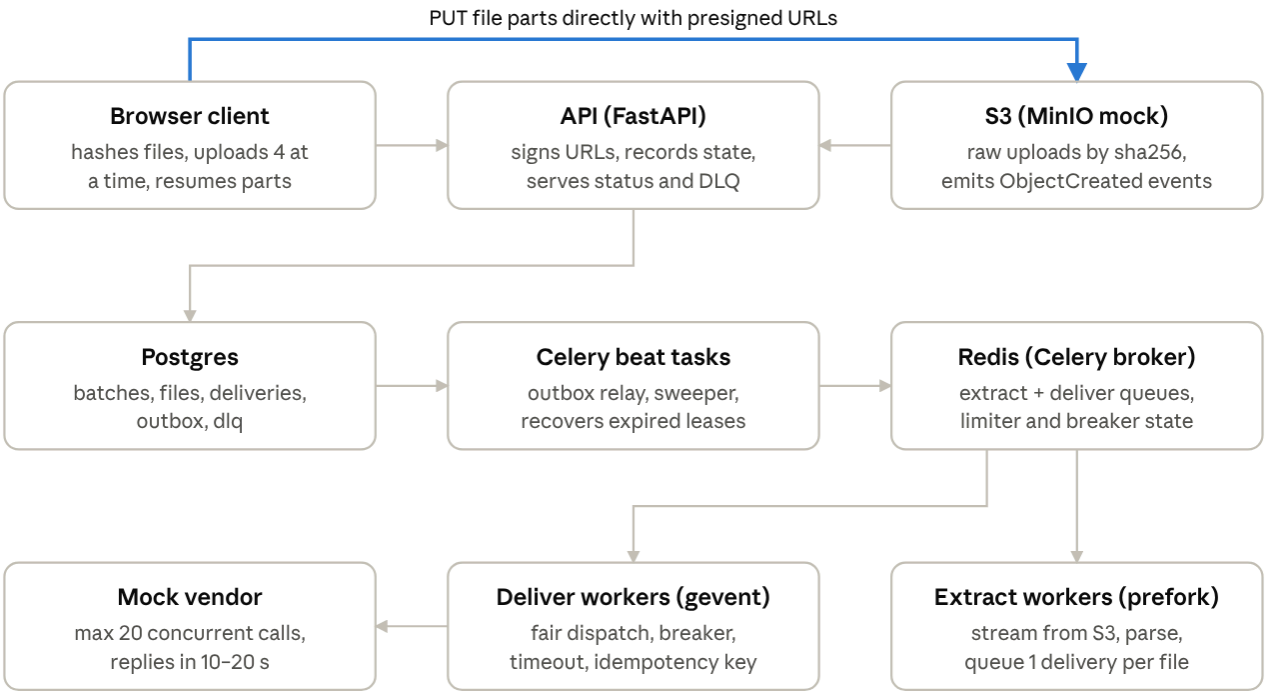
Property	What it means here	Mechanism
Concurrent	Process many files and vendor calls in parallel, bounded	Celery queues + worker pools, per-stage concurrency limits
Idempotent	Any message, upload or vendor call can repeat with no duplicate effect	Idempotency keys, content hashes, conditional DB writes, vendor idempotency headers
Rate limited	Never exceed the vendor's 20 concurrent calls	20-slot Redis semaphore shared by all workers, honours `Retry-After`
Circuit broken	Stop calling a failing vendor, recover automatically	Vendor circuit breaker (closed / open / half-open)
Graceful errors	Partial success is normal; errors are classified and retried or parked	Error taxonomy, jittered backoff, dead-letter queue, status API

Non-goals: virus scanning, user auth beyond a stub token, a production deployment pipeline, and real vendor contracts.

Architecture

The system is nine local services: the browser, a FastAPI API, mock S3, Postgres, Redis, Celery beat (outbox relay and sweeper), two Celery worker pools and the mock vendor. Bytes travel only between the browser and S3; everything else moves small state records and task ids.

File bytes go straight to S3; the API only handles state



system architecture · 9 components

The API never sees file bytes, and every hand-off between stages goes through the Postgres outbox, so a crash cannot lose or duplicate work.

End-to-end flow for a 50-file batch

1. Client creates the batch and gets 50 task ids with their upload plans (presigned URLs).
2. Client uploads parts straight to S3 and calls `complete` per file; S3 also fires `ObjectCreated`.
3. API marks the file `UPLOADED` and writes an outbox row; the beat relay task sends `extract_file` to the `extract` queue.
4. A prefork worker streams the file from S3, parses it, stores JSON in S3, and writes 1 delivery row + 1 outbox row in one transaction.

5. The fair dispatcher waits for one of the vendor's 20 slots, then sends `deliver_file` to the `deliver` queue. A gevent worker calls the vendor (10–20 s) through breaker → timeout → HTTP.
6. The result updates the delivery row; file and batch status are recomputed and pushed to the client over SSE.

Client-side direct upload

The client asks the API for presigned URLs, uploads bytes straight to S3 with at most 4 files in flight, then tells the API each file is done. The API only signs URLs and records state.

Flow

1. Client computes SHA-256 per file in a Web Worker (streamed in 8 MB chunks, so the UI never freezes).
2. `POST /batches` with an `Idempotency-Key` header (a UUID generated once per user action) and the file list `{name, size, mimeType, sha256}`. The API validates type and size, creates the batch and file rows, and returns one upload plan per file. A retried request with the same key returns the same batch.
3. Each plan is either a single presigned `PUT` (files under 16 MB) or a multipart upload: `uploadId` + presigned URLs per 8 MB part. URLs expire after 15 minutes.
4. The object key is content-addressed: `uploads/{tenantId}/{sha256}`. A file already in S3 with the same hash is marked `UPLOADED` immediately and skipped (dedupe).
5. Client uploads with a pool of 4 files and 3 parts per file. Each part retries up to 5 times with jittered backoff on network errors and 5xx. A 403 (expired URL) calls `POST /files/:id/refresh-urls`.
6. After the last byte, client calls `POST /files/:id/complete` with the part ETags. The API completes the multipart upload, runs `HeadObject` to check size and the `x-amz-checksum-sha256` value, and moves the file to `UPLOADED`.
7. S3 also emits an `ObjectCreated` event. Both the event and the `complete` call try to enqueue the same job; the conditional state update (see Idempotency) lets only one win.

Resume and abandonment

- Upload progress (`fileId`, `uploadId`, finished part numbers) is kept in IndexedDB. After a page reload the client calls `GET /files/:id/parts` (S3 `ListParts`) and uploads only missing parts.

- A sweeper aborts multipart uploads still in `UPLOADING` after 24 hours and marks the file `UPLOAD_EXPIRED`. An S3 lifecycle rule `AbortIncompleteMultipartUpload` (1 day) is the backstop.

Validation and security

- Presigned `PUT` pins `Content-Type`, `Content-Length` and `x-amz-checksum-sha256`, so a client cannot upload different bytes than it declared.
- Allowed types: PDF, CSV, XLSX, JSON. Max 100 MB per file, 50 files per batch. Rejects return `422` with a per-file reason, and valid files in the same batch still proceed.
- The bucket is private; CORS allows `PUT` from the app origin only and exposes the `ETag` header.

Data model and state machine

Postgres is the source of truth for every state. Four tables track batches, files, each file's vendor delivery, and the outbox; every state change is a conditional `UPDATE ... WHERE status = <expected>`, so concurrent workers cannot both advance the same row.

```
CREATE TABLE batches (
  id              uuid PRIMARY KEY,
  tenant_id       text NOT NULL,
  idempotency_key text NOT NULL,
  status          text NOT NULL,          -- OPEN | PROCESSING | COMPLETED |
PARTIAL | FAILED
  file_count      int  NOT NULL,
  created_at      timestamptz DEFAULT now(),
  UNIQUE (tenant_id, idempotency_key)
);

CREATE TABLE files (
  id            uuid PRIMARY KEY,
  batch_id      uuid REFERENCES batches(id),
  sha256        text NOT NULL,
  s3_key        text NOT NULL,
  size_bytes    bigint NOT NULL,
  status        text NOT NULL,            -- see file states
  attempts      int  NOT NULL DEFAULT 0,
  last_error    jsonb,
  extracted_key text,                     -- S3 key of extracted JSON
  lease_until   timestamptz,              -- worker lease for crash recovery
  version       int  NOT NULL DEFAULT 0,
  UNIQUE (batch_id, sha256)
);
```

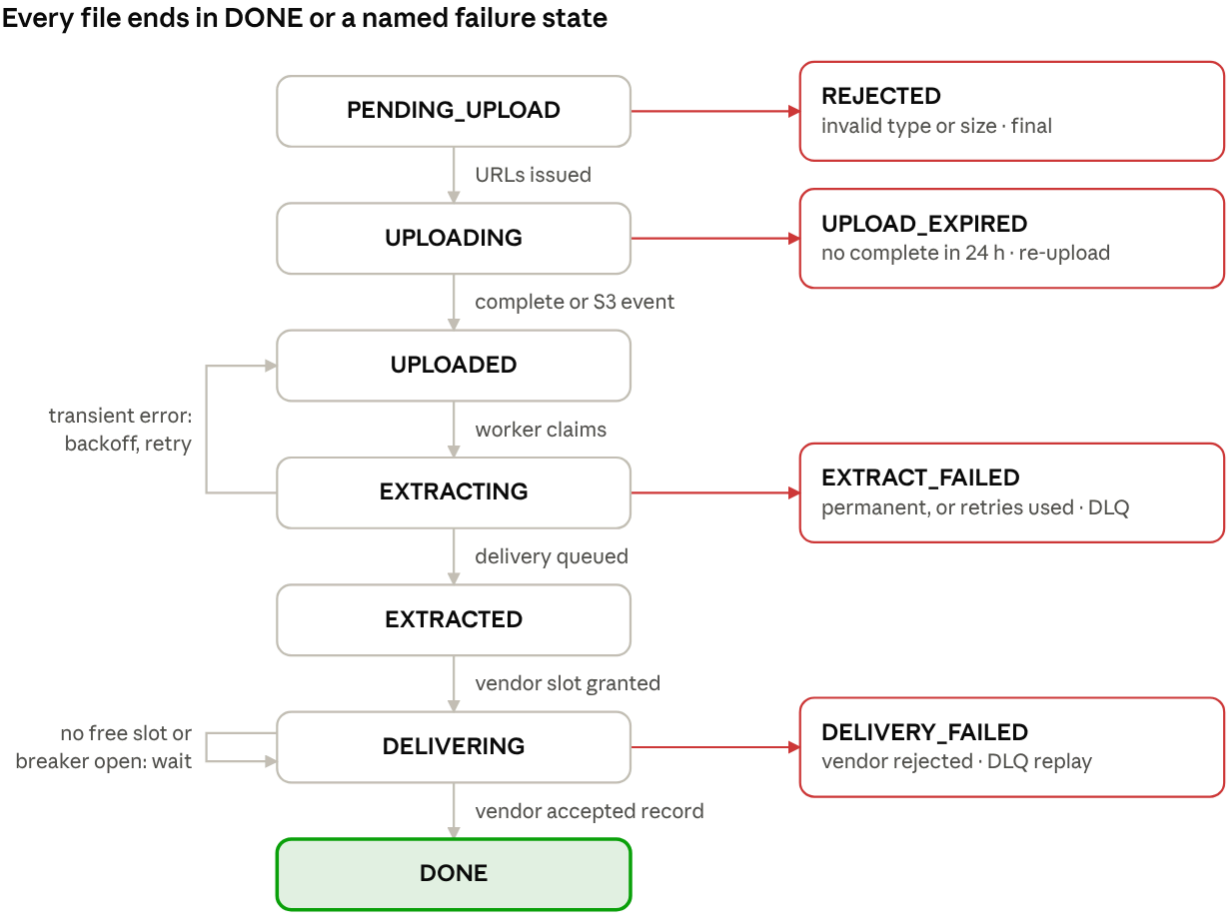
```
-- One row per file today (single vendor). Kept as its own table so a second
-- vendor later is a new row per file, not a schema change.
CREATE TABLE vendor_deliveries (
  file_id         uuid REFERENCES files(id),
  vendor          text NOT NULL DEFAULT 'vendor',
  idempotency_key text NOT NULL,          -- sha256(file.sha256 + vendor +
payload_version)
  status          text NOT NULL,          -- PENDING | IN_FLIGHT | DELIVERED
| RETRY_WAIT | FAILED
  attempts        int  NOT NULL DEFAULT 0,
  next_attempt_at timestamptz,
  vendor_ref      text,                   -- id returned by vendor
  last_error      jsonb,
  PRIMARY KEY (file_id, vendor)
);

CREATE TABLE outbox (
  id         bigserial PRIMARY KEY,
  topic      text NOT NULL,
  dedupe_key text UNIQUE NOT NULL,
  payload    jsonb NOT NULL,
  sent_at    timestamptz
);
```

File states: `PENDING_UPLOAD → UPLOADING → UPLOADED → EXTRACTING → EXTRACTED → DELIVERING → DONE`. Side exits: `REJECTED` (validation, final), `UPLOAD_EXPIRED` (final, user can re-upload), `EXTRACT_FAILED` (after retries, goes to DLQ), `DELIVERY_FAILED` (vendor rejected the record or retries ran out, goes to DLQ).

Batch status is derived after every file transition: `COMPLETED` when all files are `DONE`, `PARTIAL` when all are terminal and some failed, `FAILED` when none succeeded, otherwise `PROCESSING`.

The diagram below shows the file lifecycle.



file state machine · 7 main states, 4 failure exits

Throttling and open breakers only delay a file; they never move it to a failure state or use up its retries.

Concurrency model

Work runs on separate Celery queues with separate worker pools, so slow vendor calls never block extraction. Extraction is CPU-bound and runs in processes; delivery spends 10–20 s waiting on each call and runs in greenlets.

Stage	Celery queue	Pool	Concurrency	Unit of work
Extract	`extract`	`prefork`	2 workers × `-c 4` (about one per CPU core)	1 file
Deliver	`deliver`	`gevent`	1 worker × `-c 50`; only 20 can hold a vendor slot	1 file delivery

Stage	Celery queue	Pool	Concurrency	Unit of work
Fair dispatcher	none (long-running process)	—	1 loop	Hands out the vendor's 20 slots fairly (see Handling bursts)
Notify	`notify`	`prefork`	`-c 2`	1 notification
Outbox relay	beat task, every 1 s	—	1 (Redis lock)	Up to 100 outbox rows
Sweeper	beat task, every 60 s	—	1 (Redis lock)	Expired leases, stuck uploads

Why gevent for delivery: 20 calls × 15 s average means 20 workers sitting idle most of the time. With `prefork` that is 20 processes doing nothing; with `gevent` it is 20 greenlets in one process. httpx's sync client works under gevent's monkey-patching, and a spare 30 greenlets absorb DB and S3 work around the calls.

Celery configuration

```
# pipeline/celery_app.py
from celery import Celery
from pipeline.settings import settings

app = Celery("pipeline", broker=settings.redis_url)
app.conf.update(
    task_acks_late=True,                # ack after the task finishes, not
on receipt
    task_reject_on_worker_lost=True,    # crashed worker -> message returns
to the queue
    worker_prefetch_multiplier=1,       # a worker holds only what it is
running
    broker_transport_options={"visibility_timeout": 3600},  # longer than any
task
    task_soft_time_limit=90,            # SoftTimeLimitExceeded inside the
task
    task_time_limit=120,                # hard kill
    worker_max_memory_per_child=512_000,  # KB; recycle a bloated prefork
child
    task_routes={
        "pipeline.tasks.extract_file": {"queue": "extract"},
        "pipeline.tasks.deliver_file": {"queue": "deliver"},
        "pipeline.tasks.send_notification": {"queue": "notify"},
    },
    beat_schedule={
```

```
        "relay-outbox": {"task": "pipeline.tasks.relay_outbox", "schedule":
1.0},
        "sweep": {"task": "pipeline.tasks.sweep", "schedule": 60.0},
    },
)
```

```
celery -A pipeline.celery_app worker -Q extract -P prefork -c 4 -n extract@%h
celery -A pipeline.celery_app worker -Q deliver -P gevent -c 50 -n deliver@%h
celery -A pipeline.celery_app worker -Q notify -P prefork -c 2 -n notify@%h
celery -A pipeline.celery_app beat
python -m pipeline.dispatcher
```

Fan-out: when extraction finishes, one DB transaction marks the file `EXTRACTED`, inserts its `vendor_deliveries` row, and writes an outbox row `deliver:{file_id}`. The relay pushes the job onto the user's fair queue in Redis; the dispatcher sends it to Celery only when a vendor slot is free. A crash between DB commit and publish cannot lose or duplicate work: the relay retries and the task dedupes.

Backpressure

- The `deliver` Celery queue never holds more than 20 messages, because the dispatcher only sends when it holds a slot. Everything else waits in per-user Redis lists, where fairness and queue position are known.
- Extract workers take one message at a time (`prefetch_multiplier=1`); the rest stays in the `extract` queue.
- When the breaker is open, the dispatcher hands out no slots until the probe time. Jobs stay where they are and no retries are used.
- Long waits never use Celery `countdown` / `eta`: on a Redis broker those tasks sit in worker memory and can be redelivered after `visibility_timeout`. Delayed deliveries go to a Redis sorted set `delayed:deliver` (score = due time), which the dispatcher moves back when due. Short extract retries (up to 2 min) use `self.retry(countdown=...)`.
- If more than 5,000 deliveries are waiting, the dispatcher calls `app.control.cancel_consumer("extract")` and re-adds it with `add_consumer` once the backlog drops below 2,500. Extracted payloads live in S3, so pausing costs nothing.

Leases and crash recovery: `acks_late` + `reject_on_worker_lost` return a crashed worker's message to the queue. A worker also sets `lease_until = now() + 2 min` when it claims a row, renewed every 30 s, so a frozen (not crashed) worker is caught by the sweeper, which resets the row and re-enqueues it. On `SIGTERM` Celery does a warm shutdown and finishes running tasks; the container stop grace period is 60 s, longer than the 30 s vendor timeout.

Idempotency

Every boundary assumes at-least-once delivery, which is what Celery with `acks_late` gives, and makes repeats harmless. The rule: derive a stable key from content, then let a unique constraint or conditional update decide the single winner.

Boundary	Duplicate source	Key	Guard
Create batch	Client retry, double click	`Idempotency-Key` header	`UNIQUE (tenant_id, idempotency_key)`; on conflict return the stored response
File upload	Same file twice	`sha256` of content	Content-addressed S3 key; `UNIQUE (batch_id, sha256)`
Upload complete	`complete` call + S3 event	`file_id`	`UPDATE files SET status='UPLOADED' WHERE id=:id AND status='UPLOADING'`; 0 rows = already done
Extract task	Celery redelivery after a crash, sweeper re-enqueue	`task_id=f"extract:{file_id}"`	Claim via `UPDATE ... WHERE status='UPLOADED' RETURNING *`
Extract output	Retry after a partial write	`extracted/{sha256}/v{parser_version}.json`	Deterministic S3 key; overwrite yields the same bytes
Delivery fan-out	Relay retry	`outbox.dedupe_key = deliver:{file_id}`	Unique constraint on outbox + `vendor_deliveries` primary key
Vendor call	Timeout after the vendor accepted	`sha256(file.sha256 + vendor + payload_version)`	Sent as `Idempotency-Key` header; vendor returns the original result on repeat

Claiming a row (the first step of every task):

```
def claim_delivery(session: Session, file_id: UUID) -> Delivery | None:
    row = session.execute(
        text("""
            UPDATE vendor_deliveries
               SET status = 'IN_FLIGHT', attempts = attempts + 1
             WHERE file_id = :file_id
               AND status IN ('PENDING', 'RETRY_WAIT')
         RETURNING *
        """),
        {"file_id": file_id},
    ).mappings().one_or_none()
    session.commit()
    return Delivery(**row) if row else None   # None: someone else has it, or
it is finished
```

A task that gets `None` returns immediately, so a duplicate Celery message costs one SQL statement.

If the vendor does not honour idempotency keys: before retrying after a timeout, call its lookup endpoint (`GET /records?externalId=<key>`) and mark `DELIVERED` if the record exists. If there is no lookup, a timeout is treated as `Unknown` and routed to the DLQ for manual review rather than risking a duplicate.

Rate limiting

The vendor accepts at most 20 concurrent calls, so the core limit is a 20-slot semaphore in Redis, shared by every process. With 10–20 s responses, 20 slots deliver about 1.3 files per second (20 ÷ 15 s average), so concurrency, not request rate, is the bottleneck.

Setting	Value	Notes
`VENDOR_MAX_CONCURRENT`	20	The vendor's hard limit; size of the semaphore
`VENDOR_SLOTS`	20	Slots we actually use; set to 18 if the vendor frees slots slightly late
Vendor response time	10–20 s	Mock default
`VENDOR_TIMEOUT_S`	30	Above the slowest normal reply; anything slower counts as a failure
Slot lease	45 s	Timeout + 15 s, so a crashed worker's slot frees itself

Setting	Value	Notes
Request-rate cap	Off	Optional token bucket if the vendor also limits requests per second

All values live in `pipeline/config.py` (Pydantic settings) and can be changed with environment variables, no deploy.

Why not Celery's `rate_limit`: it limits tasks per second per worker, not concurrent calls, and does not add up across workers. Only a shared semaphore can guarantee "never more than 20 at once" across every process.

Distributed semaphore: a Redis sorted set `sem:vendor`, member = lease id, score = expiry time. Acquire is one Lua script: drop expired members, then add if `ZCARD < limit`. The dispatcher acquires a slot and passes the lease id to `deliver_file`; the task refreshes the lease when the call starts and releases it in `finally`.

```
ACQUIRE_LUA = """
local key, now, ttl, limit, lease = KEYS[1], tonumber(ARGV[1]),
tonumber(ARGV[2]), tonumber(ARGV[3]), ARGV[4]
redis.call('ZREMRANGEBYSCORE', key, '-inf', now)
if redis.call('ZCARD', key) < limit then
  redis.call('ZADD', key, now + ttl, lease)
  return 1
end
return 0
"""

class VendorSemaphore:
    def __init__(self, r: Redis, vendor: str = "vendor", lease_s: int = 45):
        self.r, self.key, self.lease_s = r, f"sem:{vendor}", lease_s
        self._acquire = r.register_script(ACQUIRE_LUA)

    def acquire(self) -> str | None:
        lease = uuid4().hex
        ok = self._acquire(keys=[self.key], args=[time.time(), self.lease_s,
effective_slots(), lease])
        return lease if ok else None

    def refresh(self, lease: str) -> None:
        self.r.zadd(self.key, {lease: time.time() + self.lease_s}, xx=True)

    def release(self, lease: str) -> None:
        self.r.zrem(self.key, lease)
```

Vendor `429` responses: these should not happen if we are the vendor's only caller, but other systems may share the account. On a `429`, read `Retry-After` (seconds or HTTP date), stop handing out slots until then, and put the job in `delayed:deliver`. Three `429`s within 60 s cut `effective_slots()` by 25% (20 → 15); it then grows back by 1 slot every 30 s up to 20. Rate-limit waits never count as retries.

Circuit breaking

The vendor has one breaker, stored in Redis and shared by the dispatcher and every worker, so an outage stops new calls within seconds and queued deliveries wait instead of failing. Thresholds are sized for about 80 calls a minute (20 slots ÷ 15 s).

State	Behaviour	Moves to
Closed	Dispatcher hands out slots; outcomes recorded in a 60 s rolling window	Open, when the window has ⩾ 20 calls and ⩾ 50% failed, or 10 consecutive failures
Open	No new slots handed out; waiting jobs stay queued	Half-open, after cooldown (30 s, doubling on each re-open, max 5 min)
Half-open	Exactly 3 probe calls allowed (Redis `INCR` gate)	Closed if all 3 succeed; Open if any fails

When the breaker opens, up to 20 calls may already be in flight. They finish or time out normally (at most 30 s); their outcomes are recorded but do not restart the cooldown.

What counts as a failure: timeouts (no reply in 30 s), connection errors, and `5xx`. **Not failures:** `4xx` validation errors (`400`, `422`), which are the payload's fault and go to the DLQ for that delivery only, and `429`, which is handled by the limiter.

Order of checks for each delivery:

1. Dispatcher checks the breaker (one Redis read). If open, it hands out nothing until the probe time.
2. Dispatcher acquires one of the 20 slots and sends `deliver_file` with the lease id.
3. Worker refreshes the slot lease and calls the vendor with a 30 s timeout.
4. Worker records the outcome on the breaker and classifies any error for the retry policy.
5. Except after a timeout (see Mocking), the worker releases the slot in `finally`.

Retries are scheduled through the dispatcher's delayed set, never in a loop inside the task, so an open breaker also pauses retries immediately.

```
def call_vendor(delivery: Delivery, payload: dict, gate: Gate) -> str:
    try:
        resp = http.post(
            f"{settings.vendor_url}/records",
            json=payload,
            headers={"Idempotency-Key": delivery.idempotency_key},
            timeout=httpx.Timeout(settings.vendor_timeout_s, connect=5.0),
        )
    except (httpx.TimeoutException, httpx.TransportError) as err:
        breaker.on_failure(gate)
        raise VendorTimeout() from err

    if resp.status_code >= 500:
        breaker.on_failure(gate)
        raise VendorUnavailable(resp.status_code)
    if resp.status_code == 429:
        raise Throttled(retry_after_s=parse_retry_after(resp))
    if resp.status_code >= 400:
        raise VendorRejected(resp.status_code, resp.json())

    breaker.on_success(gate)
    return resp.json()["id"]
```

Breaker state changes emit a metric and a log line (`breaker_state{vendor}`), and an alert fires if it stays open for over 10 minutes.

Error handling

Every error is classified into one of five kinds, and the kind alone decides what happens next. Failures are scoped to the smallest unit: one part, one file, or one file × vendor delivery.

Kind	Examples	Action	Counts toward retries
`Transient`	Timeout, `ECONNRESET`, `5xx`, S3 `SlowDown`	Retry with backoff	Yes
`Throttled`	Our limiter, vendor `429`	Re-delay by wait time / `Retry-After`	No
`CircuitOpen`	Breaker open	Re-delay to probe time	No

Kind	Examples	Action	Counts toward retries
`Permanent`	Corrupt file, unsupported format, vendor `400` / `422`	Mark failed, send to DLQ with reason	—
`Unknown`	Timeout on a vendor without idempotency or lookup	DLQ for manual review	—

Backoff: full jitter, `delay = random(0, min(cap, base × 2^attempt))`. Extract: base 2 s, cap 2 min, 5 attempts. Deliver: base 5 s, cap 10 min, 8 attempts. After the last attempt the item becomes `Permanent`.

File-level handling

- Extraction runs in a Celery `prefork` child with `soft_time_limit=60` (raises `SoftTimeLimitExceeded`, classified `Permanent: EXTRACT_TIMEOUT`) and `worker_max_memory_per_child`, so a malformed or huge file only costs that child process.
- Files are streamed from S3 (`get_object()["Body"].iter_chunks()` into the parser), never loaded fully into memory.
- Partial extraction (e.g. 3 of 1,000 CSV rows invalid) succeeds with a `warnings` list stored beside the output, instead of failing the file.
- Before extraction, the task re-checks size and checksum with `head_object`; a mismatch is `Permanent` (`CHECKSUM_MISMATCH`).

Dead-letter queue: a `dlq` table stores `{scope, id, kind, code, message, attempts, payload_ref, first_seen, last_seen}`. Celery's own failed results are not used as the DLQ; the table is the single place to look. Admin endpoints:

- `GET /dlq?scope=delivery`
- `POST /dlq/{id}/replay` — resets the row to `PENDING` and re-queues it; idempotency keys make replay safe.
- `POST /batches/{id}/retry-failed` — replays every failed item in a batch.

What the user sees: `GET /batches/{id}` returns per-file status, delivery status, and a human-readable error (`"Vendor unavailable, retrying at 14:05"`). The client follows it live over SSE (polling every 5 s as fallback) at `/batches/{id}/events`. A batch is never shown as simply "failed": it shows counts such as "47 done, 2 retrying, 1 rejected (unsupported type)".

Error model used in the DB, API and logs:

```
class ErrorKind(StrEnum):
    TRANSIENT = "Transient"
    THROTTLED = "Throttled"
    CIRCUIT_OPEN = "CircuitOpen"
    PERMANENT = "Permanent"
    UNKNOWN = "Unknown"

class PipelineError(BaseModel):
    kind: ErrorKind
    code: str                      # e.g. VENDOR_TIMEOUT, CHECKSUM_MISMATCH
    message: str                   # safe to show the user
    stage: Literal["upload", "extract", "deliver"]
    retry_at: datetime | None = None

def classify(err: Exception) -> PipelineError:
    match err:
        case Throttled():           return
PipelineError(kind=ErrorKind.THROTTLED, ...)
        case CircuitOpen():         return
PipelineError(kind=ErrorKind.CIRCUIT_OPEN, ...)
        case VendorTimeout() | VendorUnavailable() | botocore_retryable():
                                    return
PipelineError(kind=ErrorKind.TRANSIENT, ...)
        case VendorRejected() | ChecksumMismatch() | UnsupportedFormat() |
SoftTimeLimitExceeded():
                                    return
PipelineError(kind=ErrorKind.PERMANENT, ...)
        case _:                     return
PipelineError(kind=ErrorKind.TRANSIENT, code="UNEXPECTED", ...)
```

The raw exception and traceback go to logs only, never to the API.

Task tracking, progress and notifications

Every file gets a task id the moment the batch is created, before any byte is uploaded. Every state change of that task is written to an append-only event log in the same transaction, and pushed live to the user. The user can follow a single task, the whole batch, or ask to be notified when the batch finishes.

Task ids

- `POST /batches` returns `batchId` (`bat_<ULID>`) and one `taskId` (`tsk_<ULID>`) per file. ULIDs sort by creation time and are safe to show to users.
- A retried `POST /batches` with the same `Idempotency-Key` returns the same task ids.

- A task covers the file's full life: upload, extraction and the vendor delivery, each shown as a stage of the task.

```
{
  "batchId": "bat_01JC8Z6Q4M2X",
  "tasks": [
    { "taskId": "tsk_01JC8Z6Q5A1B", "fileName": "invoice-001.pdf", "status":
"PENDING_UPLOAD",
      "upload": { "multipart": true, "uploadId": "...", "partUrls": ["..."] }
}
  ],
  "links": { "batch": "/batches/bat_01JC8Z6Q4M2X", "events":
"/batches/bat_01JC8Z6Q4M2X/events" }
}
```

Additions to the data model

```
ALTER TABLE files   ADD COLUMN task_id text UNIQUE NOT NULL;      --
tsk_<ULID>
ALTER TABLE files   ADD COLUMN progress smallint NOT NULL DEFAULT 0; -- 0-100
ALTER TABLE batches ADD COLUMN user_id text NOT NULL;
ALTER TABLE batches ADD COLUMN notify jsonb;                       --
channels requested
ALTER TABLE batches ADD COLUMN completed_at timestamptz;

CREATE TABLE task_events (
  id          bigserial PRIMARY KEY,       -- also the SSE event id
  task_id     text NOT NULL,
  batch_id    uuid NOT NULL,
  at          timestamptz NOT NULL DEFAULT now(),
  stage       text NOT NULL,               -- upload | extract | deliver |
notify
  vendor      text,
  from_status text,
  to_status   text NOT NULL,
  attempt     int,
  error_kind  text,                        -- Transient | Throttled |
CircuitOpen | Permanent | Unknown
  error_code  text,
  message     text,                        -- safe to show the user
  next_retry_at timestamptz
);
CREATE INDEX ON task_events (task_id, id);
CREATE INDEX ON task_events (batch_id, id);

CREATE TABLE notifications (
```

```
  batch_id uuid NOT NULL,
  channel  text NOT NULL,                  -- email | webhook | in_app
  status   text NOT NULL,                  -- PENDING | SENT | FAILED
  attempts int NOT NULL DEFAULT 0,
  sent_at  timestamptz,
  PRIMARY KEY (batch_id, channel)          -- at most one notification per
channel
);
```

Progress model

Progress is a number from 0 to 100 per task, computed from stage weights so it only moves forward. Batch progress is the average over its tasks, plus counts per state.

Stage	Progress range	Source of progress
Upload	0–40%	Parts completed / total parts (client shows byte-level progress locally)
Extract	40–60%	40 when claimed, 60 when JSON is stored
Deliver	60–100%	60 when queued, 75 when the vendor call starts, 100 when it reaches a terminal state (`DELIVERED` or `FAILED`)

Tracking endpoints

Endpoint	Returns
`GET /tasks/:taskId`	Status, stage, progress, delivery status, queue position, last error, next retry time, allowed actions
`GET /tasks/:taskId/history`	Full event timeline from `task_events` , oldest first
`GET /batches/:batchId`	Batch status, progress, counts per state, task list (paginated)
`GET /batches/:batchId/events`	Server-Sent Events stream of every task event in the batch
`GET /tasks?status=FAILED&since=...`	The user's tasks across batches, filterable by state

Live updates: each transaction that writes a `task_events` row also writes an outbox row; the relay publishes it to Redis channel `batch:{batchId}`. API nodes subscribe and push it on the SSE stream. The SSE event id is the `task_events.id`, so a reconnecting browser sends `Last-Event-ID` and the API replays missed events from Postgres. Nothing is lost if the tab sleeps or the network drops. If SSE is blocked, the client polls `GET /batches/:id` every 5 s.

Tracking a failure

`GET /tasks/:taskId` always answers what failed, where, why, and what happens next:

```
{
  "taskId": "tsk_01JC8Z6Q5A1B",
  "status": "DELIVERY_FAILED",
  "progress": 100,
  "stages": {
    "upload":  { "status": "DONE", "finishedAt": "2026-10-30T14:01:12Z" },
    "extract": { "status": "DONE", "warnings": 2, "finishedAt": "2026-10-
30T14:01:20Z" },
    "deliver": {
      "status": "FAILED",
      "attempts": 1,
      "error": {
        "kind": "Permanent",
        "code": "VENDOR_REJECTED",
        "message": "The vendor rejected the record: field 'taxId' is missing"
      }
    }
  },
  "actions": ["retry_delivery"],
  "history": "/tasks/tsk_01JC8Z6Q5A1B/history"
}
```

What the user sees	Meaning	Action offered
Waiting for vendor: position 37, about 30 s	All 20 slots busy, not an error	None needed
Sending to vendor (started 12 s ago)	Call in flight; replies take 10–20 s	None needed
Retrying, next attempt 14:05 (attempt 3 of 8)	Timeout or `5xx`, backoff running	None needed
Vendor unavailable, paused until 14:07	Circuit breaker open	None needed

What the user sees	Meaning	Action offered
Rejected: unsupported file type	Validation failed at upload	Upload a different file
Extraction failed: file is corrupt	Permanent extract error	Re-upload
Vendor rejected the record: reason	Permanent vendor error, in DLQ	Retry after fixing data

Error codes and user-facing messages live in one catalogue (`pipeline/errors/catalog.py`), so the API, UI and notifications use the same wording.

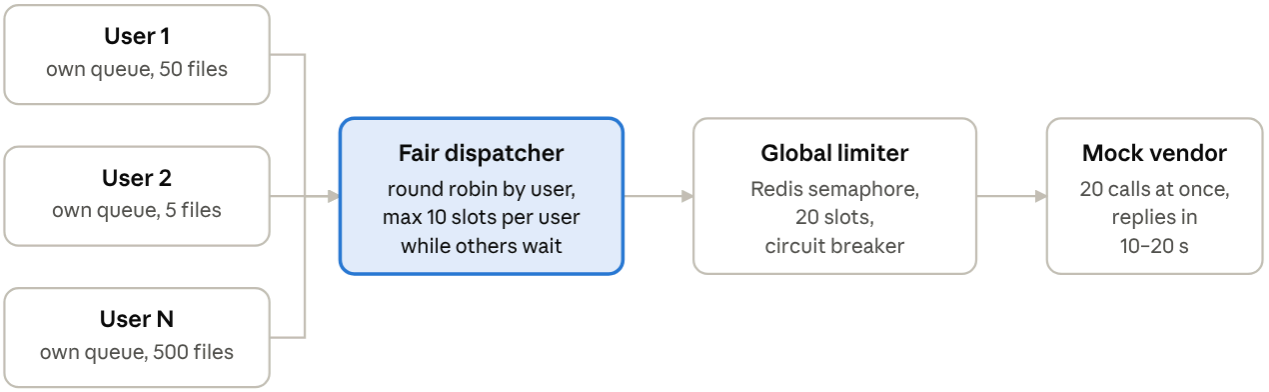
Notifications when all files are processed

- The user opts in when creating the batch (`"notify": {"channels": ["in_app", "email"], "webhookUrl": "..."}`) or later with `PUT /batches/:id/notify`.
- When the last task reaches a terminal state, the transaction that sets the batch to `COMPLETED`, `PARTIAL` or `FAILED` also inserts one `notifications` row and one outbox row per channel. The primary key makes duplicates impossible, even if two workers finish the last two files at the same moment.
- The send_notification Celery task sends each one: in-app via the SSE stream and a browser notification (Notifications API, with permission), email via SMTP, webhook as a signed POST (`X-Signature: HMAC-SHA256`). Failed sends retry with backoff up to 6 times.
- The message carries counts, duration and failed tasks: *"Batch bat_01JC8Z…: 48 of 50 files processed. 2 need attention: invoice-017.pdf (corrupt file), invoice-031.pdf (vendor rejected record)."*
- Optional `"notifyOn": ["first_failure"]` sends an early alert when any task fails permanently.
- Local mocks: MailHog catches email (web UI on `:8025`); a small webhook receiver in `mock-vendor` records webhook calls for tests.

Handling bursts from many users

A burst is absorbed by queues, never rejected because the vendor is busy. The vendor's 20-slot limit is enforced globally in Redis, so 1 user or 100 users produce the same peak load: exactly 20 calls in flight. A fair dispatcher shares those slots between users, so one 500-file batch cannot starve a 5-file batch.

Many users share the vendor's 20 slots fairly



burst handling · per-user queues, fair dispatch, global vendor limits

Each user has their own delivery queue in Redis; the dispatcher takes jobs from them in turn and sends one to Celery only when it holds one of the 20 slots.

Protection by layer

Layer	Protects	Mechanism	Limit (default)
API admission	API, Postgres	Per-user token bucket in Redis	10 batch creates/min, 1,000 active tasks per user; beyond that `429` with `Retry-After`
Upload	S3	Direct browser → S3, no server cost	S3 limits only
Extract queue	CPU	Celery `extract` queue; workers scale with `--autoscale=8,2` (KEDA in production)	2–8 processes per worker
Deliver dispatch	Fairness	Per-user Redis lists, round robin	One user holds at most 10 of the 20 slots while others wait; all 20 when alone
Vendor limiter	Vendor	Redis semaphore + breaker	20 concurrent calls

Fair dispatcher (`pipeline/dispatcher.py`)

- Waiting jobs live in Redis lists `q:deliver:{user_id}` ; a sorted set `active_users` holds users with waiting jobs, and a hash `inflight` counts each user's slots.

- One dispatcher runs at a time, holding a leader lock (`SET dispatcher:leader NX PX 10000` , renewed every 3 s); a standby takes over within 10 s. The semaphore is global, so even two dispatchers could not exceed 20.
- For each free slot it picks the next user in round-robin order, skips users already at 10 slots when others wait, and sends `deliver_file` to Celery.
- Delayed jobs (backoff, `Retry-After` , breaker) sit in `delayed:deliver` and return to their user's list when due.

```
def dispatch_loop(stop: threading.Event) -> None:
    while not stop.is_set():
        fair_queue.promote_due_delayed()            # delayed:deliver -> user
lists
        gate = breaker.allow()                      # None while open;
"closed" or "probe"
        if gate is None:
            stop.wait(breaker.seconds_until_probe())
            continue
        lease = semaphore.acquire()                 # one of the 20 slots
        if lease is None:
            stop.wait(0.2)                          # all 20 busy
            continue
        job = fair_queue.pop_next()                 # round robin, max 10
slots per user
        if job is None:
            semaphore.release(lease)
            stop.wait(0.2)
            continue
        deliver_file.apply_async(
            args=[job.file_id, lease, gate],
            task_id=f"deliver:{job.file_id}:{job.attempt}",
        )
```

Continuous processing, no visible interruption

- Jobs are never dropped or failed because of load. Slot waits, `429` s and breaker waits only delay; they do not use retries.
- Queues are durable: Redis with AOF persistence, Postgres as the source of truth. The sweeper re-queues any delivery whose row says `PENDING` but has no job in Redis.
- Users see that work is moving: queued tasks show their position and an estimate, e.g. *"Waiting for vendor: position 37, about 30 s"*. The estimate is position ÷ measured throughput (moving average over the last 5 min, about 1.3 files/s).
- Extraction keeps running while delivery is the bottleneck, so files are ready the moment a slot frees up.

- Deploys use Celery warm shutdown, so a release does not pause anyone's batch.

Capacity (20 slots, replies in 10–20 s, 15 s on average → about 1.33 files/s)

Scenario	Files	Time to deliver all (fast to slow vendor)
1 user, one batch	50	about 45 s (3 rounds of 20, 20, 10; 30–60 s)
5 users at once	250	about 3 min (2–4 min)
20 users at once	1,000	about 12.5 min (8–17 min)
Small batch arriving during the 1,000-file burst	5	under 1 min, thanks to round robin

The vendor's 20 slots set the finish time, which is why completion notifications matter: in a burst, users can leave and be told when their batch is done. The burst test in Testing runs the 20-user case and asserts that calls in flight never exceed 20.

Mocking S3 and the vendor

S3 is replaced by MinIO, which speaks the real S3 API, so presigned URLs, multipart uploads and bucket notifications work unchanged from the browser. The vendor is a small FastAPI app that enforces its own 20-call limit and answers each call after a random 10–20 s delay, with faults you can switch on at runtime.

Mock S3 (MinIO)

- `docker compose` runs `minio/minio` on `:9000` with bucket `uploads` and CORS for `http://localhost:5173`.
- Bucket notification `s3:ObjectCreated:*` posts to the API webhook `POST /internal/s3-events`, mirroring S3 → SQS/EventBridge in production.
- The app uses boto3 with `endpoint_url="http://localhost:9000"` and path-style addressing; switching to real S3 is config only.
- Unit tests use moto (`@mock_aws`) instead of MinIO for speed.

Mock vendor (`mock_vendor` service, port 4000)

Routes: `POST /records`, `GET /records?externalId=`, `GET /health`, `PUT /_control` to change behaviour, `GET /_control/stats`. It runs as a single Uvicorn process so its in-flight counter is exact.

```
# mock_vendor/app.py
app = FastAPI()
cfg = MockConfig()          # latency_s=(10, 20), max_concurrent=20,
error_rate=0.05, ...
stats = Stats()
records: dict[str, dict] = {}
inflight = 0

@app.post("/records")
async def create_record(body: dict, idempotency_key: str = Header(...)):
    global inflight
    if cfg.honor_idempotency and idempotency_key in records:
        stats.duplicates_seen += 1
        return records[idempotency_key]             # same answer, no new
record
    if cfg.outage:
        raise HTTPException(503, "vendor down")
    if inflight >= cfg.max_concurrent:
        stats.rejected_429 += 1                     # our limiter should
keep this at 0
        return JSONResponse({"error": "too many concurrent calls"}, 429,
headers={"Retry-After": "2"})

    inflight += 1
    stats.peak_inflight = max(stats.peak_inflight, inflight)
    try:
        await asyncio.sleep(cfg.rng.uniform(*cfg.latency_s) *
cfg.latency_scale)   # 10-20 s
        roll = cfg.rng.random()
        if roll < cfg.timeout_rate:
            await asyncio.sleep(40 * cfg.latency_scale)   # past the client's
30 s timeout
        elif roll < cfg.timeout_rate + cfg.error_rate:
            raise HTTPException(500, "random failure")
        elif roll < cfg.timeout_rate + cfg.error_rate + cfg.reject_rate:
            raise HTTPException(422, {"field": "taxId", "error": "missing"})
        record = {"id": f"V-{uuid4().hex[:8]}", "externalId":
idempotency_key}
        records[idempotency_key] = record
        stats.accepted += 1
        return record
    finally:
        inflight -= 1
```

Knob	Default	Purpose
`latency_s`	`[10, 20]`	Random response time per call
`latency_scale`	`1.0`	Multiplies every delay; CI uses `0.01` (0.1–0.2 s) so suites finish fast
`max_concurrent`	`20`	Returns `429` when exceeded, so our limiter is tested against a real limit
`error_rate`	`0.05`	Random `500`s
`timeout_rate`	`0.02`	Replies after 40 s, past our 30 s timeout
`reject_rate`	`0.01`	`422` validation errors
`outage`	`false`	All requests `503`; drives the breaker open
`honor_idempotency`	`true`	Dedupes by `Idempotency-Key` and returns the first result
`seed`	random	Fixed seed makes fault patterns reproducible

`GET /_control/stats` returns received, accepted, duplicates seen, `429`s and peak in-flight calls. Tests assert peak in-flight ⩽ 20, zero `429`s, and zero duplicate records.

A timed-out call can still be running at the vendor. The mock's 40 s timeout case shows this: our client gives up at 30 s, but the vendor still counts the call. So after a timeout the worker does not release its slot; it lets the 45 s lease expire. That keeps us at 20 calls by the vendor's count, not just ours.

Example chaos script:

```
# take the vendor down for 60 s while a 50-file batch runs
curl -X PUT localhost:4000/_control -H 'content-type: application/json' -d
'{"outage": true}'
sleep 60
curl -X PUT localhost:4000/_control -H 'content-type: application/json' -d
'{"outage": false}'
curl localhost:4000/_control/stats
```

Project layout and key code

One Python package (`pipeline`) holds the API, Celery tasks, dispatcher and shared resilience code, so every process enforces the same rules. The browser client stays in React + TypeScript, because uploading from the browser has to run in JavaScript.

```
file-pipeline/
├─ docker-compose.yml        # postgres, redis, minio, mailhog, mock_vendor,
api, workers, beat, dispatcher, flower
├─ pyproject.toml            # uv; ruff, mypy, pytest config
├─ web/                      # React client: hashing, upload pool, progress
view
|  └─ src/upload/{hashWorker.ts, uploadManager.ts, multipart.ts,
resumeStore.ts}
├─ pipeline/
|  ├─ api/                   # FastAPI routers: batches, files, tasks, dlq,
s3 webhook, SSE
|  ├─ celery_app.py
|  ├─ tasks/{extract.py, deliver.py, notify.py, relay.py, sweep.py}
|  ├─ dispatcher.py          # fair dispatcher process
|  ├─ resilience/{semaphore.py, breaker.py, fair_queue.py, backoff.py}
|  ├─ errors/{model.py, catalog.py}
|  ├─ extract/{csv.py, pdf.py, xlsx.py, json.py}
|  ├─ db/{models.py, repo.py}
|  ├─ vendor_client.py
|  └─ config.py              # Pydantic settings
├─ alembic/
├─ mock_vendor/app.py
└─ tests/{unit/, integration/, e2e/, chaos/, load/}
```

Client upload manager (bounded pool, resumable parts):

```
export async function uploadBatch(files: File[]) {
  const hashes = await Promise.all(files.map(f => hashInWorker(f)));
  const batch = await api.createBatch(
    files.map((f, i) => ({ name: f.name, size: f.size, mimeType: f.type,
sha256: hashes[i] })),
    { idempotencyKey: crypto.randomUUID() },
  );
  const filePool = pLimit(4);
  await Promise.allSettled(batch.files.map((plan, i) => filePool(async () =>
{
    if (plan.status === 'UPLOADED') return;            // deduped by hash
    try {
      const etags = plan.multipart
        ? await uploadParts(files[i], plan, { partConcurrency: 3, retries: 5
})
        : await putWithRetry(plan.url, files[i], { retries: 5 });
      await api.completeFile(plan.fileId, etags);
    } catch (err) {
      ui.markFileFailed(plan.fileId, err);              // other files
continue
    }
  })));
}
```

Extract task:

```
@app.task(bind=True, acks_late=True, max_retries=5)
def extract_file(self, file_id: str) -> None:
    with session_scope() as s:
        file = repo.claim_file(s, file_id, frm="UPLOADED", to="EXTRACTING")
    if file is None:
        return                                          # already handled
    try:
        verify_object(file)                             # head_object: size
+ checksum
        out = run_extractor(file)                       # streams from S3
        out_key = f"extracted/{file.sha256}/v{PARSER_VERSION}.json"
        s3.put_object(Bucket=BUCKET, Key=out_key, Body=out.to_json_bytes())
        with session_scope() as s:                      # one transaction
            repo.mark_extracted(s, file.id, out_key, out.warnings)
            repo.insert_delivery(s, file.id, delivery_key(file))
            repo.outbox(s, "deliver", f"deliver:{file.id}",
                        {"file_id": str(file.id), "user_id": file.user_id})
    except Exception as exc:
        err = classify(exc)
        if err.kind is ErrorKind.TRANSIENT and self.request.retries <
```

```
self.max_retries:
            with session_scope() as s:
                repo.release_file(s, file.id, back_to="UPLOADED", error=err)
            raise self.retry(exc=exc,
countdown=full_jitter(self.request.retries, base=2, cap=120))
        with session_scope() as s:
            repo.fail_file(s, file.id, "EXTRACT_FAILED", err)   # + DLQ row +
task event
```

Deliver task:

```
@app.task(bind=True, acks_late=True)
def deliver_file(self, file_id: str, lease: str, gate: str) -> None:
    keep_slot = False                                       # True after a
timeout: let the lease expire
    with session_scope() as s:
        d = repo.claim_delivery(s, file_id)
    if d is None:
        semaphore.release(lease)
        return
    try:
        semaphore.refresh(lease)
        payload = load_payload(d)                           # extracted JSON
from S3
        vendor_ref = call_vendor(d, payload, gate)          # waits 10-20 s
        with session_scope() as s:
            repo.mark_delivered(s, d, vendor_ref)
    except VendorTimeout as exc:
        keep_slot = True
        handle_delivery_failure(d, classify(exc))
    except Exception as exc:
        handle_delivery_failure(d, classify(exc))           # delayed set with
backoff, or DLQ
    finally:
        if not keep_slot:
            semaphore.release(lease)
        fair_queue.slot_done(d.user_id)
        recompute_file_and_batch_status(file_id)            # may queue
completion notifications
```

Delivery retries do not use `self.retry`: a retry has to wait for a free slot like any other job, so `handle_delivery_failure` puts it in the dispatcher's delayed set with jittered backoff (base 5 s, cap 10 min, 8 attempts), or in the DLQ when the error is permanent or attempts run out.

Testing and observability

The acceptance test is one command: upload 50 files through the real client against MinIO and the mock vendor with faults on and real 10–20 s delays, then assert every file reaches a terminal state, no record is duplicated, and the vendor never sees more than 20 calls at once. That run takes about a minute; the larger suites use `latency_scale` to shrink delays.

Test levels

Level	Tools	Covers	Vendor delay
Unit	pytest, freezegun, fakeredis, moto	Breaker transitions, semaphore Lua script, fair queue order, backoff, `classify()`, extractors	None (mocked client)
Integration	pytest + testcontainers (Postgres, Redis, MinIO), `celery.contrib.pytest` worker	Claim races, outbox relay, idempotent `complete`, DLQ replay, dispatcher	`latency_scale=0.01`
End-to-end	Playwright + `docker compose`	Browser upload of 50 files, resume after reload, progress and error UI	Real 10–20 s
Load	Locust	20 users × 50 files; queue drain time, fairness, peak in-flight	`latency_scale=0.1`

Chaos scenarios (each is an automated test)

- [ ] 50 files, vendor healthy → batch `COMPLETED`, 50 deliveries, 0 duplicates, peak in-flight exactly 20.
- [ ] Vendor outage for 60 s mid-batch → breaker opens, closes after recovery, every delivery ends `DELIVERED`.
- [ ] `SIGKILL` a deliver worker mid-call → message redelivered (`acks_late`), mock shows 0 duplicate records, slot freed by lease expiry.
- [ ] Kill the dispatcher leader → standby takes over within 10 s, no slot leaked.

- [ ] 10% of calls time out (reply at 40 s) → slots held until lease expiry, mock `rejected_429` stays 0.
- [ ] Same file uploaded twice in one batch and across batches → stored once, extracted once.
- [ ] Client retries `POST /batches` 3 times → one batch, same task ids.
- [ ] Browser reload at 60% of a 100 MB upload → only missing parts uploaded.
- [ ] 2 corrupt files + 1 unsupported type in 50 → batch `PARTIAL`, 47 `DONE`, each failed task shows stage, cause and action.
- [ ] Duplicate S3 event + `complete` call → one extract task.
- [ ] DLQ replay of a delivery that actually succeeded → no duplicate (idempotency key).
- [ ] Burst: 20 users upload 50 files each at once → no `429` to users, vendor in-flight never above 20, all 1,000 tasks finish, a 5-file user finishes within 1 min (at real delays).
- [ ] SSE disconnect for 30 s mid-batch → reconnect with `Last-Event-ID` replays every missed event, progress never goes backwards.
- [ ] Last two files finish on two workers at the same moment → exactly one completion notification per channel.

Metrics (Prometheus)

- `pipeline_files_total{stage,status}`, `pipeline_stage_duration_seconds{stage}`
- `vendor_inflight` (must stay ⩽ 20), `vendor_latency_seconds` (expect 10–20 s), `vendor_requests_total{outcome}`
- `deliver_waiting_total`, `deliver_waiting_by_user{user}`, `slot_wait_seconds`
- `breaker_state` (0 closed, 1 half-open, 2 open), `dlq_size{scope}`
- Celery queue lengths and task runtimes via `celery-exporter`; Flower on `:5555` for live worker views

Logs and traces: structlog JSON with `batch_id`, `task_id`, `file_id`, `celery_task_id`, `attempt`, `error_code` on every line. OpenTelemetry instrumentation for FastAPI, Celery and httpx links spans from `POST /batches` through extraction to the vendor call, so a slow call shows up clearly in a trace.

Alerts: breaker open > 10 min, DLQ growth > 20 in 15 min, oldest waiting delivery > 15 min, vendor p95 latency > 25 s, and `vendor_inflight` > 20 (should never fire).

Implementation plan and open questions

The build runs in 7 milestones, each ending in a demo you can run locally. Resilience pieces land before the vendor worker uses them, so nothing ships without limits.

1. **Local stack and skeleton** — `docker compose` (Postgres, Redis, MinIO, MailHog, mock vendor, Flower), uv project, Alembic migrations, Celery app with queues and routing. Demo: services start, health checks green, a no-op task runs on each queue.
2. **Direct upload and task ids** — FastAPI batch endpoint with idempotency key and task ids, presigned single and multipart URLs, client hashing and upload pool, `complete` + S3 webhook, resume from IndexedDB. Demo: 50 files land in MinIO from the browser, each with a task id.
3. **Extraction and task events** — `extract_file` on the prefork pool, claim pattern, streaming extractors, checksum check, outbox + beat relay, `task_events` on every transition. Demo: every file reaches `EXTRACTED` with a full history.
4. **Resilience package** — 20-slot semaphore, breaker, fair queue, backoff, `classify()`, error catalogue, all unit tested with freezegun and fakeredis.
5. **Delivery and fairness** — dispatcher with leader lock, `deliver_file` on the gevent pool, vendor client, mock vendor with 10–20 s delays, DLQ and replay endpoints. Demo: 50 deliveries with faults on, 0 duplicates, peak in-flight 20.
6. **Tracking and notifications** — task and batch endpoints, SSE with replay, progress UI with failure causes, completion notifications (in-app, email, webhook). Demo: a batch with failures notifies once and every cause is visible.
7. **Hardening** — chaos and burst tests, Locust load test, metrics, dashboards, alerts, sweeper and graceful shutdown.

Open questions

- [ ] Is the vendor's limit 20 **concurrent** calls (this doc's assumption) or 20 calls per second or minute? A rate limit would switch on the token bucket instead.
- [ ] Is the 20-call limit shared with other systems using the same vendor account? If so, how many slots can this pipeline use?
- [ ] Does the vendor support idempotency keys or a lookup endpoint?
- [ ] What data is extracted per file type, and what payload shape does the vendor expect?
- [ ] Retention: how long do raw uploads and extracted JSON stay in S3?
- [ ] Which notification channels are needed first (in-app, email, webhook), and should paid tiers get a higher fair-share weight?